

Supplementary Material for “BeatEdit: Symbolic Music Generation as Explicit Editing”

Haoyu Gu
School of Future Technology
South China University of Technology
Guangzhou, China
ghy20050104@gmail.com

Lekai Qian
School of Future Technology
South China University of Technology
Guangzhou, China
ftqlk@mail.scut.edu.cn

Haowu Zhou
School of Future Technology
South China University of Technology
Guangzhou, China
202364870491@mail.scut.edu.cn

Qi Liu[✉]
School of Future Technology
South China University of Technology
Guangzhou, China
drliuqi@scut.edu.cn

Shuai Wang[✉]
School of Intelligence Science and
Technology
Nanjing University
Suzhou, China
shuaiwang@nju.edu.cn

Abstract

This document contains the supplementary material for “BeatEdit: Symbolic Music Generation as Explicit Editing”. It provides the complete encoding specification, edit label space, pre-training and implementation details, inference algorithms, a worked end-to-end example, full per-scheme results, ablation studies, statistical significance tests, subjective evaluation details, efficiency analysis, and multi-track generalization details. Section letters here match the “Appendix X” references in the main paper.

A Encoding Technical Specification

A.1 Complete Token Vocabulary

Table 1: Complete token ID allocation for separated encoding (Schemes A, B). Under bundled encoding (Schemes C, D), position and pattern tokens merge into composite tokens (IDs 0–7,128), with separate control tokens.

Token Type	ID Range	Description
Pattern values	0–80	Ternary rhythm patterns ($3^4 = 81$)
Position markers	81–168	81 + pitch_index (max 87)
EMPTY_MARKER	169	Empty beat (single token)
BAR	170	Bar separator
EOS	171	End of sequence
BOS	172	Beginning of sequence
PAD	173	Padding token
TIME_SIG	174–178	5 time signatures (4/4, 3/4, 2/4, 6/8, 2/2)
BPM	179–182	4 tempo categories
T0_START	183	Right-hand voice marker
T1_START	184	Left-hand voice marker
MASK	185	MLM pre-training only

A.2 Ternary Pattern Semantics

The within-beat state vector $\mathbf{s}_p \in \{0, 1, 2\}^\tau$ of pitch p (0 = silent, 1 = onset, 2 = sustain) is mapped to a single pattern token by base-3

conversion:

$$s_p = \sum_{t=0}^{\tau-1} s_p[t] \cdot 3^{\tau-1-t} \in \{0, \dots, 3^\tau - 1\}. \quad (1)$$

For $\tau = 4$ this yields 81 values; e.g. a quarter note (1, 2, 2, 2) $\mapsto 1 \cdot 27 + 2 \cdot 9 + 2 \cdot 3 + 2 = 53$ and two eighth notes (1, 2, 1, 2) $\mapsto 50$. Selected pattern values for $\tau = 4$:

Ternary s_p	s_p	Musical Meaning
(1, 2, 2, 2)	53	Quarter note (onset + sustain)
(1, 2, 1, 2)	50	Two eighth notes
(1, 0, 0, 0)	27	Sixteenth note (staccato)
(0, 0, 1, 2)	5	Note on beat subdivision 3
(2, 2, 2, 2)	80	Sustained from previous beat
(0, 0, 0, 0)	0	Silent (not encoded)

A.3 Encoding and Decoding Algorithms

B Edit Label Space

Table 2: Complete 350-label edit space for separated encoding (Schemes A, B). Under bundled encoding, REPLACE and APPEND target 7,128 music tokens and SHIFT is absent, yielding 14,258 labels.

Operation	Label ID	Count	Computation
KEEP	0	1	—
DELETE	1	1	—
REPLACE _{v}	$2 + v$	169	$v \in [0, 168]$
APPEND _{v}	$171 + v$	169	$v \in [0, 168]$
SHIFT _{$+n$}	$339 + n$	5	$n \in [1, 5]$
SHIFT _{$-n$}	$344 + n $	5	$n \in [-5, -1]$

C Music BERT Pre-Training Details

The Music BERT encoder uses the architecture specified in Table 3. Pre-training uses standard MLM: 15% of music tokens are selected, of which 80% are replaced with [MASK], 10% with a random music

Algorithm 1 BEAT Encoding (Single-Track)

Require: piano-roll $X \in \{0, 1, 2\}^{P \times T}$, steps per beat τ , beats per bar n

Ensure: Token sequence E

```

1:  $N \leftarrow \lceil T/\tau \rceil$  {Number of beats}
2:  $E \leftarrow []$ 
3: for  $i = 1$  to  $N$  do
4:   if  $(i - 1) \bmod n = 0$  then
5:     Append BAR to  $E$ 
6:   end if
7:   Append BEAT to  $E$ 
8:    $B^{(i)} \leftarrow X[:, (i-1)\tau : i\tau]$  {Beat segment}
9:    $\mathcal{A} \leftarrow \{p : \exists t, B^{(i)}[p, t] \neq 0\}$  {Active pitches}
10:  if  $\mathcal{A} = \emptyset$  then
11:    Append EMPTY_MARKER to  $E$  and continue
12:  end if
13:  Sort  $\mathcal{A}$  ascending:  $p_1 < p_2 < \dots < p_M$ 
14:  for  $j = 1$  to  $M$  do
15:     $s_{p_j} \leftarrow B^{(i)}[p_j, :]$  {State vector}
16:     $s_{p_j} \leftarrow \text{BASE3ToINT}(s_{p_j})$  {Pattern token}
17:    if  $j = 1$  then
18:       $d_1 \leftarrow p_1$  {Absolute pitch}
19:    else
20:       $d_j \leftarrow p_j - p_{j-1}$  {Relative interval}
21:    end if
22:    Append (PIT  $d_j$ , PAT  $s_{p_j}$ ) to  $E$ 
23:  end for
24: end for
25: return  $E$ 

```

Table 3: Music BERT architecture and pre-training configuration.

Component	Configuration
Layers / Heads / Hidden / FFN	8 / 8 / 512 / 2,048
Parameters	~26.6M
Vocab size	185 (184 + MASK)
Max position embeddings	2,048
Training data	192,788 pieces (154K / 19.3K / 19.3K split)
Masking	15% of music tokens
Batch size / Epochs	256 effective / 30
Optimizer / Schedule	AdamW, lr 10^{-4} , cosine + 10% warmup

token, and 10% left unchanged. Control tokens (IDs ≥ 169) are never masked. Data augmentation applies random transposition (± 5 semitones) with 70% probability; sequences exceeding 2,048 tokens are randomly truncated.

Training dynamics. Best validation losses (across all epochs) vary across encoding schemes: Scheme B achieves the lowest loss (0.224, perplexity 1.25), followed by A (0.282, ppl 1.33), C (0.449, ppl 1.57), and D (0.502, ppl 1.65). The lower perplexity of separated schemes reflects their smaller vocabulary (185 vs. 7,145). Training times range from 4.6h (Scheme D) to 27.3h (Scheme B) on 2 GPUs (24GB VRAM each).

Algorithm 2 BEAT Decoding (Single-Track)

Require: Token sequence E , steps per beat τ

Ensure: piano-roll $X \in \{0, 1, 2\}^{P \times T}$

```

1: Initialize  $X \leftarrow \mathbf{0}^{P \times T}$ 
2:  $i \leftarrow 0$ ,  $p_{\text{prev}} \leftarrow \text{NONE}$ 
3: for each token in  $E$  do
4:   if token is BAR then
5:     Continue
6:   else if token is BEAT then
7:      $i \leftarrow i + 1$ ,  $p_{\text{prev}} \leftarrow \text{NONE}$ 
8:   else if token is EMPTY_MARKER then
9:     Continue
10:  else if token is (PIT  $d$ , PAT  $s$ ) then
11:    if  $p_{\text{prev}} = \text{NONE}$  then
12:       $p \leftarrow d$  {Absolute pitch (first in beat)}
13:    else
14:       $p \leftarrow p_{\text{prev}} + d$  {Relative pitch}
15:    end if
16:    if  $0 \leq p < P$  then
17:       $X[p, (i-1)\tau : i\tau] \leftarrow \text{INTToBASE3}(s, \tau)$ 
18:    end if
19:     $p_{\text{prev}} \leftarrow p$ 
20:  end if
21: end for
22: return  $X$ 

```

D Implementation Details**D.1 Head and Loss Formulations**

Let $\mathbf{h}_i \in \mathbb{R}^d$ denote the shared encoder’s hidden representation at position i . All prediction heads are single linear layers over $\mathbf{h}_i$.

SeqTag (§3.2 of the main paper). The binary error-detection head and the multi-class tag head are

$$\hat{e}_i = \sigma(\mathbf{W}_{\text{det}} \mathbf{h}_i + \mathbf{b}_{\text{det}}), \quad \hat{t}_i = \text{softmax}(\mathbf{W}_{\text{tag}} \mathbf{h}_i + \mathbf{b}_{\text{tag}}), \quad (2)$$

trained jointly with $\mathcal{L} = \mathcal{L}_{\text{tag}} + \lambda \cdot \mathcal{L}_{\text{detect}}$, where $\mathcal{L}_{\text{tag}}$ is weighted cross-entropy with down-weighted KEEP and $\mathcal{L}_{\text{detect}}$ is binary cross-entropy. At inference $\hat{t}_i$ is overridden to KEEP wherever $\hat{e}_i < \theta$.

IterEdit (§3.3). The deletion, insertion, and token heads are

$$\hat{d}_i = \sigma(\mathbf{W}_{\text{del}} \mathbf{h}_i), \quad \hat{n}_j = \text{softmax}(\mathbf{W}_{\text{ins}} [\mathbf{h}_j; \mathbf{h}_{j+1}]), \quad \hat{x}_i = \text{softmax}(\mathbf{W}_{\text{tok}} \mathbf{h}_i), \quad (3)$$

where $\hat{n}_j$ predicts the number of placeholders (0–20) to insert in the gap j between adjacent tokens. The joint loss $\mathcal{L}_{\text{iter}} = \mathcal{L}_{\text{del}} + \mathcal{L}_{\text{ins}} + \mathcal{L}_{\text{tok}}$ sums three cross-entropy terms; token prediction uses label smoothing ($\epsilon=0.1$).

TagFill (§3.4). The tagger is trained with Focal Loss

$$\mathcal{L}_{\text{tagger}} = - \sum_i (1 - \hat{p}_{i, y_i})^\gamma \log \hat{p}_{i, y_i}, \quad \gamma = 2.0, \quad (4)$$

and the inserter commits $\hat{x}_i = \arg \max_v P_{\text{ins}}(v | \tilde{X})$ for $i \in \mathcal{M}_t$, the confidence-ranked subset of masks accepted at pass t .

D.2 Configurations**E SeqTag Inference Algorithm**

Table 4: SeqTag configuration.

Component	Configuration
<i>Heads</i>	
Error detector	Linear(512, 2)
Tag predictor	Linear(512, 350 / 14,258)
Dropout	0.1
<i>Training</i>	
Cold start	2 epochs, freeze BERT (lr 10^{-3})
Full fine-tune	18 epochs, patience 3
Fine-tune lr (BERT / heads)	10^{-5} / 10^{-4}
Batch size / Precision	64 effective / fp16
KEEP weight / λ_{detect}	0.15 / 0.5
<i>Inference</i>	
Max iterations	3
KEEP bias β / Error threshold θ	0.3 / 0.5

Table 5: TagFill (tag-then-fill) configuration.

Component	Configuration
<i>Tagger</i>	
Architecture	8-layer Transformer, 512 hidden
Output head	Linear(512, 11)
Loss	Focal Loss ($\gamma = 2.0$)
<i>Insertter</i>	
Architecture	8-layer Transformer, 512 hidden
Output head	Linear(512, V)
Label smoothing	0.1
MaskGIT passes T	2
<i>Training (both stages)</i>	
Optimizer / Schedule	AdamW, lr 10^{-4} , cosine + 10% warmup
Weight decay / Epochs	0.01 / 30
Batch size	96 effective (32×3 accum.)
Perturbation levels	L1:30%, L2:30%, L3:25%, L4:15%

Algorithm 3 Iterative music error correction (SeqTag)

Require: Encoded tokens X , model $\mathcal{M}$, max rounds $R_{\max} = 3$

- 1: **for** $r = 1$ to $R_{\max}$ **do**
- 2: $d \leftarrow \mathcal{M}_{\text{detect}}(X)$ {binary error detection}
- 3: $t \leftarrow \mathcal{M}_{\text{tag}}(X)$ {edit tag prediction}
- 4: Gate: $t_i \leftarrow \text{KEEP}$ wherever $d_i < 0.5$ {conservative KEEP bias}
- 5: Apply labels: $X \leftarrow \text{APPLYEDITS}(X, t)$
- 6: $X \leftarrow \text{DECODE-FILTER-RENCODE}(X)$ {validity check}
- 7: **if** all $t_i = \text{KEEP}$ **then**
- 8: **break**
- 9: **end if**
- 10: **end for**
- 11: **return** X

The DECODE-FILTER-RENCODE step operates beat by beat: it decodes each beat’s tokens into (pitch, pattern) notes, discards invalid ones (out-of-range pitches or pattern values, duplicate pitches within a beat), and re-encodes the remainder in ascending pitch

Table 6: IterEdit (iterative deletion-insertion) configuration.

Component	Configuration
<i>Architecture</i>	
Encoder	8-layer Transformer, 512 hidden, 8 heads
Activation / Norm	GELU / Pre-norm
Deletion head	Linear(512, 2)
Insertion head	Linear(1024, 21)
Token head	Linear(512, V)
<i>Training</i>	
Optimizer / Schedule	AdamW, lr 3×10^{-4} , cosine + 10% warmup
Weight decay / Epochs	0.01 / 30
Batch size	64 effective (32×2 accum.)
Loss	$\mathcal{L}_{\text{del}} + \mathcal{L}_{\text{ins}} + \mathcal{L}_{\text{tok}}$
Token label smoothing	0.1
<i>Inference</i>	
Max iterations	10 (completion) / 3 (editing)
Deletion threshold	0.5

order, which also repairs any ordering violations. A beat that fails to parse is left unchanged, and beats outside the edited region re-assemble verbatim, so the cycle guarantees a well-formed input for the next iteration without perturbing untouched content.

F End-to-End Error Correction Walkthrough

We trace a correction involving two error types—a pitch shift and a missing note—through the full pipeline under Scheme B (relative, separated). Consider a beat whose target is C4–E4–G4 as quarter notes (pattern token 53, Appendix A). The input has C4 shifted to D4 (+2 semitones) and G4 missing.

Step 1: Encoding. The target and corrupted input are:

Target	[120][53] [85][53] [84][53]
	(C4, E4, G4 as quarter notes)
Input	[122][53] [83][53]
	(D4, E4 only—G4 missing)

D4 yields position $81 + 41 = 122$; the relative distance from D4 to E4 is 2 (token 83).

Step 2: Iteration 1—pitch correction via SHIFT. The model detects the pitch error and predicts:

Token	[122]	[53]	[83]	[53]
Label	SHIFT ₋₂	KEEP	SHIFT ₊₂	APPEND ₈₄
Result	[120]	[53]	[85]	[53] [84]

SHIFT₋₂ corrects D4→C4; SHIFT₊₂ fixes the cascading interval to E4; APPEND₈₄ inserts a position token for the missing G4 ($81 + 3 = 84$, relative to E4). Since APPEND attaches only *one* token per anchor, the pattern token for G4 cannot be produced within the same round.

Step 3: Post-processing and convergence. The DECODE-FILTER-RENCODE step (Appendix E) parses the beat into complete (position, pattern) pairs; the trailing token [84] lacks a pattern token, does not form a valid note, and is discarded during re-encoding, yielding [120][53] [85][53]. Both pitch errors are therefore fixed within a single iteration, while the missing G4 is *not* restored: even if a later iteration proposes the same APPEND, post-processing removes the incomplete insertion again and the no-progress check

terminates inference. This reflects a structural limitation of separated encodings—restoring a deleted note requires two coordinated tokens, but validity filtering only admits complete pairs—and is consistent with note deletion being the hardest error type for Schemes A and B (Figure 1). Under bundled encoding the same repair is a single edit: in Scheme C the missing G4 corresponds to one composite token ($3 \times 81 + 53 = 296$), so APPEND₂₉₆ completes the insertion within one iteration and passes post-processing unchanged. This walkthrough demonstrates that SHIFT and APPEND cooperate within a single iteration, and that DECODE-FILTER-RENCODE acts as a structural safety net: every intermediate sequence remains well-formed and decodable, at the cost of discarding insertions left incomplete at the end of an iteration.

G Synthetic Perturbation Details

Table 7: Perturbation types for synthetic error data. Per-beat, mutually exclusive.

Type	Operation	Prob.	Constraint
Pitch shift	$p_j \leftarrow p_j + \text{Uniform}(\pm 1-3)$	0.10	Valid range
Rhythm change	$s_j \leftarrow \text{Random}([0, 80] \setminus \{s_j\})$	0.05	—
Note deletion	Remove one (p_j, s_j) pair	0.03	Keep ≥ 1 note
Note insertion	Add $(p_{\text{new}}, s_{\text{new}})$	0.02	No duplicates
No change	Identity	0.80	—

H IterEdit Inference Algorithm

Algorithm 4 Iterative editing inference (IterEdit)

Require: Current tokens X , editable flags E , model M , max rounds

```

 $R_{\max} = 10$ 
1: for  $r = 1$  to  $R_{\max}$  do
2:   // Step 1: Delete low-confidence tokens
3:    $d \leftarrow M_{\text{del}}(X)$ ; remove  $X_i$  where  $d_i > 0.5$  and  $E_i = \text{True}$ 
4:   // Step 2: Insert placeholders at editable gaps
5:    $n \leftarrow M_{\text{ins}}(X)$ ; insert  $n_j$  PLH tokens at gap  $j$  if gap is editable

6:   Mark all new PLH positions as editable
7:   // Step 3: Fill placeholders
8:    $X_i \leftarrow \arg \max M_{\text{tok}}(X)_i$  for each PLH position  $i$ 
9:   if no deletions, no insertions, and no fills then
10:    break
11:  end if
12: end for
13: return  $X$ 

```

I Generative Baseline Details

The main text reports summary results for the two generative baselines; here we provide full per-scheme and per-configuration breakdowns that reveal encoding sensitivity patterns consistent with those observed for edit-based methods.

I.1 Autoregressive LLM Baseline

To quantify the gap between autoregressive generation and edit-based paradigms, we evaluate four LLaMA-based strategies (~162M parameters, trained from scratch on the same data), ordered by the amount of positional guidance provided:

- **AR-Prompt:** takes the first 2 bars of the source as prompt and autoregressively generates the rest; the generated melody is replaced with the source melody post hoc.
- **AR-Teacher:** melody beats are fed via teacher-forcing with explicit track markers; *all* accompaniment beats are regenerated.
- **AR-Detect:** a two-stage pipeline that uses SeqTag’s error detection head to identify perturbed beats (any accompaniment token with $P(\text{error}) \geq 0.5$ flags the beat), then regenerates only those beats via LLaMA with melody teacher-forcing. This provides the fairest AR comparison, as it uses the same localization signal available to edit methods.
- **AR-Selective:** same teacher-forcing as Teacher, but only the accompaniment beats at oracle-known changed_beat_indices are regenerated; unperturbed beats retain the source content. This is the most favorable setting for AR.

Implementation. For Scheme A, melody beats start with Track0 (183) and accompaniment with Track1 (184); Teacher/Selective explicitly insert 184 before sampling accompaniment content. For bundled schemes (C/D), SPLIT_0 (7129) and SPLIT_1 (7130) serve as track markers. For Scheme B (no explicit marker), beats alternate by position and terminate with END_MARKER (170). Prompt/Full post-processing parses the generated sequence into beat pairs and replaces melody beats with source melody.

Fairness note. Edit methods (SeqTag/TagFill/IterEdit) see the complete corrupted source and must autonomously identify which positions need modification. AR-Prompt and AR-Teacher do not know which beats are perturbed and regenerate all accompaniment. AR-Selective additionally receives oracle changed_beat_indices—information unavailable to edit methods. Results for Selective therefore represent an *upper bound* on AR performance under ideal localization, not a fair head-to-head comparison.

Table 8: LLaMA baseline results ($n = 200$, perturbed-only). Best per task—strategy in bold. [†]Selective receives oracle changed-beat positions.

Strategy	Task	Sch.	beat [†]	nf1 [†]	MPE [↓]	FMD [↓]
AR-Select. [†]	Corr.	A	.477	.637	8.6	1.17
	Edit.	A	.499	.603	9.6	0.85
	Compl.	A	.397	.512	14.4	0.39
AR-Detect	Corr.	A	.394	.650	9.1	1.24
	Edit.	A	.278	.478	15.2	1.73
AR-Teacher	Corr.	B	.062	.238	20.0	5.74
	Edit.	A	.101	.211	25.6	7.07
	Compl.	C	.117	.221	22.6	5.82
AR-Prompt	Corr.	D	.093	.242	16.3	11.2
	Edit.	A	.095	.211	25.8	3.89
	Compl.	A	.152	.221	18.7	1.54

The results reveal a clear *localization hierarchy*. Prompt and Teacher, which lack positional guidance, achieve at most 0.152 beat

exact match. Detect, using SeqTag’s error detector for automated localization, jumps to 0.394 on correction and 0.278 on editing—a substantial improvement but still below edit methods (SeqTag: 0.726, IterEdit: 0.480). Selective, given oracle positions, reaches 0.477–0.499 but still cannot match per-token precision on correction (0.477 vs. 0.726). Detect is inapplicable to completion, as the error detector cannot identify missing content.

The gap between Detect and edit methods is particularly informative: both use the same localization signal (SeqTag’s detector), but edit methods apply targeted token-level operations while Detect regenerates entire beats via AR sampling. This confirms that the editing mechanism itself—not just localization—is the source of the performance advantage. Selective consistently favors Scheme A, and its FMD values (0.39–1.17) are competitive with edit methods, suggesting that AR generation can produce distributionally natural output at the cost of requiring oracle positional guidance.

J.2 Discrete Diffusion Baseline

We evaluate D3PM [1] discrete diffusion (~34M parameters) with an SDEdit-style noise-then-denoise strategy at three noise ratios: $r=0.3$ (30 denoising steps), $r=0.5$ (50 steps), and $r=0.7$ (70 steps). All models are trained on the same data with all four encoding schemes ($n = 200$, perturbed-only).

Three findings are notable. First, *performance degrades sharply with noise ratio*: $r=0.3$ is the only competitive setting; $r=0.5$ halves beat exact match, and $r=0.7$ collapses to near-zero on separated encodings (FMD >200, fail rate >93%). This confirms that SDEdit requires minimal noise injection for editing—heavy noise destroys the structural information that denoising cannot recover. Second, *encoding dependence is extreme and mirrors edit methods*: bundled schemes (C/D) outperform separated schemes (A/B) by 4–8× on correction ($r03$: Scheme D 0.390 vs. A 0.089). Separated encoding distributes each note across two tokens; the diffusion process must independently recover both tokens in a consistent pair, a coordination problem that bundled encoding avoids. This parallel to edit methods’ encoding sensitivity provides independent evidence that encoding choice is a universal design lever, not method-specific. Third, *completion exposes diffusion’s fundamental limitation*: Diffusion $r03$ maintains non-trivial beat exact match on completion (0.080) but low $nf1$ (0.228), and at higher noise ratios ($r=0.5$) $nf1$ collapses to 0.002, indicating that the denoising process recovers rhythmic structure but struggles with pitch content. TagFill’s $nf1$ of 0.557 on the same task confirms that explicit generation mechanisms are required for content synthesis.

J Cross-Encoding Pre-Training Analysis

Table 3 (main paper) reports a *recovery* metric defined as $(nf1_{no-edit}) / (1 - nf1_{no-edit})$, measuring the fraction of the no-edit error gap closed by the method. BEAT achieves ~5× higher recovery than alternative encodings. Here we trace this gap to its root cause: encoding structure determines pre-training quality, which cascades into downstream editing performance.

J.1 MLM Pre-Training Convergence

All four BEAT schemes converge to PPL = 1.29–1.83, with relative-separated (Scheme B) achieving the lowest perplexity (1.29). Bundled schemes (C/D) start from much higher initial PPL due to their 7,145-token vocabulary but undergo a sharp phase transition around epoch 12–18, ultimately converging within 0.2–0.5 of separated schemes. Among alternative encodings, REMI and Structured converge normally (PPL = 1.39, 1.57), while MIDI-Like plateaus at 8.18–5–6× higher—reflecting its flat event-stream structure where interleaved NoteOn/NoteOff tokens across all 128 pitches create long-range dependencies that MLM cannot capture. CPWord’s apparent PPL of ~500 reflects its compound vocabulary structure (joint prediction over 5 factored sub-vocabularies) rather than a failure to learn; its downstream EditF1 confirms meaningful learning.

K Full Per-Scheme Results

Tables 11 and 12 present the complete per-scheme breakdown for all method–task combinations (perturbed-only, $n = 200$). The main text (Tables 3 and 4) reports only the best-scheme result per method; these appendix tables reveal the full encoding sensitivity. IterEdit was evaluated across all four schemes. SeqTag cannot perform segment completion (no generation head).

Several patterns emerge. *Encoding sensitivity*: Scheme B remains the weakest for TagFill and IterEdit, but SeqTag now achieves its highest correction score on Scheme B (0.726), surpassing all other schemes. TagFill and IterEdit still exhibit sharp B-column drops on correction (0.061, 0.070) and editing (0.089, 0.069), suggesting that multi-pass and generative methods are more sensitive to relative-separated encoding than the single-pass classification approach of SeqTag. The optimal encoding is method-dependent: SeqTag favors Scheme B for correction (0.726) and Scheme C for editing (0.408), TagFill favors Scheme A for correction (.368) and completion (.436) but Scheme D for editing (.350), and IterEdit is task-dependent (A for correction, D for editing). *IterEdit encoding sensitivity*: IterEdit’s correction on Scheme A (.719) far exceeds Scheme D (.569) and C (.412), a 0.307 gap—the largest encoding effect observed—yet for editing the pattern reverses (D: .480 > A: .458).

L Ablation Studies

L.1 IterEdit Architecture and Training Ablation

Table 13 compares IterEdit training configurations and architectural ablations on editing and correction (perturbed-only, $n = 200$). The “full” variants use the multi-level perturbation strategy (Section 3.3 of the main paper); “vanilla” uses the original contiguous-masking configuration; “Enc-Dec” replaces the encoder-only architecture with an encoder-decoder cross-attention design.

Three ablation findings are notable. First, adding track-aware attention bias *degrades* both editing (0.448 → 0.428, −4.5%) and correction (0.719 → 0.666, −7.4%), confirming that shared attention already captures inter-track dependencies without explicit structural bias. Second, the 4× gap between full and vanilla configurations on editing beat (.480 vs. .114) demonstrates that multi-level perturbation training is essential for IterEdit to generalize beyond contiguous masking to fine-grained editing. Third, replacing the

Table 9: Diffusion baseline results across noise ratios and encoding schemes ($n = 200$, perturbed-only). Best scheme per ratio-task in bold.

Ratio	Sch.	Error Correction				Accomp. Editing				Segment Completion			
		beat	nf1	FMD	fail	beat	nf1	FMD	fail	beat	nf1	FMD	fail
$r=0.3$	A	.093	.190	14.1	.78	.076	.134	18.20	.82	.053	.168	14.9	.99
	B	.048	.124	17.4	.87	.075	.088	22.08	.80	.050	.119	19.6	.97
	C	.340	.515	2.03	.07	.200	.372	2.75	.22	.080	.228	52.7	.65
	D	.390	.602	0.87	.04	.198	.411	1.72	.23	.080	.228	37.5	.65
$r=0.5$	A	.029	.013	217	.98	.081	.010	221	.98	.028	.000	222	.99
	B	.018	.005	171	.99	.070	.010	172	.95	.005	.000	173	1.0
	C	.153	.294	3.90	.37	.095	.186	5.91	.50	.071	.002	4.79	.68
	D	.188	.358	3.32	.29	.105	.230	5.43	.43	.074	.002	4.84	.70
$r=0.7$	A	.020	.002	453	1.0	.101	.000	454	1.0	.013	.000	455	1.0
	B	.015	.000	355	.99	.085	.000	353	.93	.003	.000	359	1.0
	C	.057	.133	7.92	.76	.046	.085	10.9	.79	.060	.002	10.2	.79
	D	.067	.191	10.2	.67	.063	.113	14.5	.71	.061	.003	11.1	.78

Table 10: MLM pre-training convergence across encodings. Values are eval perplexity ($\downarrow$). PPL values are not directly comparable across encodings with different vocabulary structures (see text).

Encoding	Vocab	Ep. 6	Ep. 12	Ep. 18	Ep. 24	Ep. 30
BEAT (Abs, Sep)	186	14.98	1.70	1.46	1.40	1.39
BEAT (Rel, Sep)	185	8.77	1.47	1.33	1.30	1.29
BEAT (Rel, Bun)	7,145	60.08	2.47	1.77	1.66	1.63
BEAT (Abs, Bun)	7,145	86.35	20.81	2.29	1.88	1.83
REMI	284	16.49	2.70	1.51	1.40	1.39
Structured	317	3.61	2.21	1.74	1.60	1.57
CPWord [‡]	306*	544	509	504	500	501
MIDI-Like [†]	403	20.54	11.43	9.06	8.43	8.18

*CPWord uses 5 compound sub-vocabs ($6 \times 38 \times 156 \times 37 \times 69$); PPL reflects joint prediction space and is not comparable to flat-vocabulary encodings.

[†]BERT MLM converged to substantially higher PPL (8.18 vs. ~ 1.5).

[‡]CPWord PPL reflects compound vocabulary (see text).

encoder-only architecture with an encoder-decoder design (Enc-Dec, 66.5M vs. 33.9M parameters) *collapses* performance (editing: .051, correction: .034) with catastrophic FMD (> 18), confirming that the encoder-only inductive bias—where edited and context tokens share attention—is critical for the editing paradigm.

L.2 Pre-training Ablation

Table 14 isolates the contribution of BERT pre-training by comparing three SeqTag configurations on Scheme D ($n = 200$, perturbed-only): the full pipeline (BERT pre-trained + fine-tuned), S1-only (BERT pre-trained, no task-specific S3 fine-tuning), and Scratch (randomly initialized, no pre-training).

Two findings emerge. First, removing BERT pre-training causes catastrophic degradation: Scratch achieves only 0.195 correction beat (-72% vs. Full) and 0.157 editing beat (-60%), confirming that domain-specific pre-training—not the SeqTag architecture itself—is

the primary driver of performance. Second, the S1-only variant slightly *exceeds* the full pipeline (correction: 0.719 vs. 0.700; editing: 0.401 vs. 0.390), suggesting that task-specific fine-tuning provides marginal benefit when the pre-trained representations are already well-suited to the editing task. Combined with the IterEdit encoder-decoder collapse above, these results identify two critical design choices: (1) domain-specific BERT pre-training, and (2) encoder-only architecture with shared attention over edited and context tokens.

L.3 Per-Error-Type Analysis

Figure 1 breaks down SeqTag correction performance by perturbation type across encoding schemes. Note insertion is easiest (Edit F1 > 0.87 for all schemes), as inserted notes create distinctive token patterns that the tagger readily detects. Rhythm changes are also well-handled (0.80–0.86). Pitch shifts are harder (0.32–0.67), especially for bundled Scheme C where a single-token pitch change requires selecting from the full 7K vocabulary. Note deletion is the most challenging (0.14–0.39), as the tagger must identify missing content from context alone.

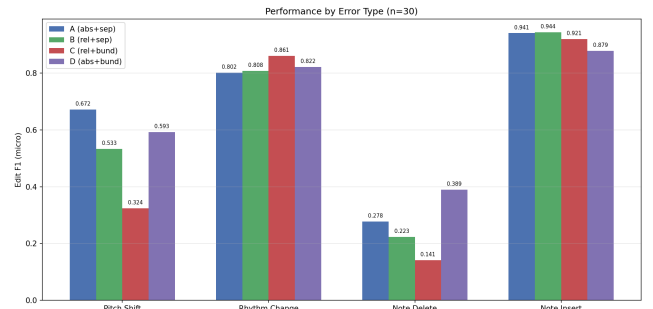
**Figure 1: SeqTag correction Edit F1 by error type across encoding schemes ($n = 30$ per type).**

Table 11: Beat exact match across all encoding schemes, methods, and tasks ($n = 200$). Best scheme per method–task cell in bold; “—” = not evaluated or not applicable.

Method	Error Correction				Accomp. Editing				Segment Completion			
	A	B	C	D	A	B	C	D	A	B	C	D
No-Edit	.030	.030	.030	.030	.037	.037	.037	.037	.000	.000	.000	.000
Copy-Ctx	.123	.129	.097	.097	.127	.127	.094	.094	.116	.067	.002	.002
BERT-CMLM	.240	.222	.327	.388	.134	.146	.186	.188	.111	.050	.000	.000
SeqTag	.700	.726	.702	.700	.382	.394	.408	.399	—	—	—	—
IterEdit	.719	.070	.412	.569	.458	.069	.352	.480	.120	.060	.011	.005
TagFill	.368	.061	.335	.329	.321	.089	.301	.350	.436	.264	.298	.326
Diffusion	.093	.048	.340	.390	.076	.075	.200	.198	.053	.050	.080	.080
AR-Prompt	.091	.069	.091	.093	.095	.087	.089	.084	.152	.105	.044	.043
AR-Detect	.394	.194	.306	.264	.278	.172	.239	.212	—	—	—	—
AR-Select. [†]	.477	.216	.216	.171	.499	.262	.236	.203	.397	.068	.240	.146
Anticipatory [‡]	—	—	—	—	—	—	—	—	.069	—	—	—

[‡]Anticipatory operates on raw MIDI; Scheme A results shown for reference (encoding-independent).

Table 12: Fréchet Music Distance (FMD, ↓) across all encoding schemes, methods, and tasks ($n = 200$). Best scheme per method–task cell in bold.

Method	Error Correction				Accomp. Editing				Segment Completion			
	A	B	C	D	A	B	C	D	A	B	C	D
No-Edit	2.89	5.33	6.79	2.69	2.83	3.74	4.50	2.48	3.05	1.04	1.76	1.95
Copy-Ctx	4.14	7.74	9.28	3.90	3.80	5.48	6.97	3.62	2.62	3.80	3.13	2.81
BERT-CMLM	0.88	2.05	2.86	1.01	1.22	1.50	1.32	0.97	2.22	3.88	2.53	2.35
SeqTag	0.08	0.15	0.32	0.09	1.86	1.87	2.56	1.58	—	—	—	—
IterEdit	1.03	8.20	4.09	1.40	1.52	3.52	2.00	1.19	0.34	1.04	2.49	2.39
TagFill	2.03	3.81	4.02	1.78	0.99	2.53	0.83	1.24	0.27	0.52	0.42	0.34
Diffusion	14.1	17.4	2.03	0.87	18.20	22.08	2.75	1.72	14.9	19.6	52.7	37.5
AR-Prompt	5.56	11.8	14.2	11.2	3.89	6.42	7.14	6.90	1.54	7.17	6.65	8.23
AR-Detect	1.24	3.97	3.34	1.62	1.73	3.13	2.33	1.56	—	—	—	—
AR-Select. [†]	1.17	3.79	4.93	2.36	0.11	1.93	1.79	1.18	0.39	1.20	0.70	1.03
Anticipatory [‡]	—	—	—	—	—	—	—	—	5.69	—	—	—

Table 13: IterEdit variant comparison (beat exact match / FMD, $n = 200$). Values may differ slightly from the full per-scheme results due to different perturbation configurations used in ablation runs.

Variant	Sch.	Editing	Correction
Full (multi-level perturb.)	D	.480 / 1.19	.568 / 1.40
Full (multi-level perturb.)	A	.448 / 2.52	.719 / 1.03
Full + track-aware bias	A	.428 / 2.54	.666 / 1.10
Full (multi-level perturb.)	C	.341 / 2.61	.412 / 4.09
vanilla (thr. = 0.3)	D	.114 / 2.71	—
vanilla (edit mode)	D	—	.129 / 2.27
Enc-Dec (cross-attn)	D	.051 / 18.5	.034 / 24.3

Table 14: SeqTag pre-training ablation (Scheme D, $n = 200$, perturbed-only). Bold: best per task.

Variant	Task	beat↑	nf1↑	FMD↓
Full (BERT + fine-tune)	Corr.	.700	.852	0.09
S1-only (BERT, no S3)	Corr.	.719	.861	0.07
Scratch (random init)	Corr.	.195	.660	1.45
Full (BERT + fine-tune)	Edit.	.390	.537	2.34
S1-only (BERT, no S3)	Edit.	.401	.540	2.27
Scratch (random init)	Edit.	.157	.451	—

L.4 Sequence Length Effect

Figure 2 shows how sequence length affects correction performance. Short sequences (<500 tokens) are harder for separated encodings (A, B), likely due to limited context. Performance stabilizes for medium and long sequences across all schemes, with bundled Scheme C maintaining the highest Edit F1 throughout.

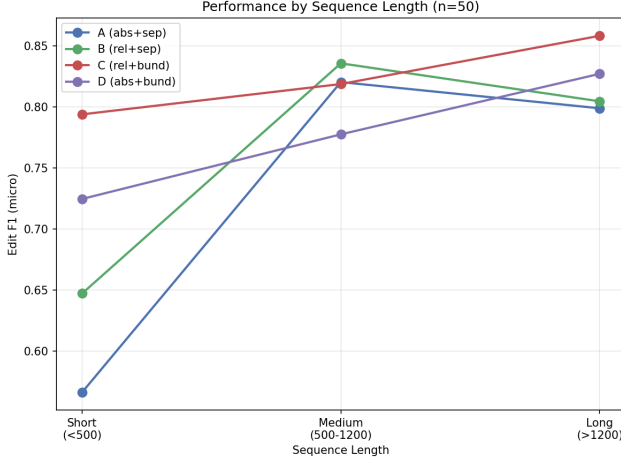


Figure 2: SeqTag correction Edit F1 by sequence length across encoding schemes ($n = 50$ per bin).

M Statistical Significance Tests

All significance tests use paired bootstrap resampling ($B = 10,000$) and Wilcoxon signed-rank tests on beat exact match ($n = 200$, perturbed-only). Effect sizes are reported as Cohen’s d (S: small, M: medium, L: large).

M.1 Pairwise Method Comparisons

The key finding is that IterEdit and SeqTag are *statistically equivalent* on correction at their respective best schemes ($p = 0.489$), while IterEdit is *significantly* better on editing across all evaluated schemes ($p < 0.001$). All other pairwise comparisons are also highly significant ($p < 0.001$).

M.2 Encoding $\times$ Method Interaction (ANOVA)

All encoding $\times$ method interactions are highly significant ($p < 0.001$), formally confirming that method rankings reverse across encoding schemes. The interaction effect is largest for correction ($\eta^2 = 12.5\%$), driven primarily by TagFill and IterEdit’s sharp drops on Scheme B (0.061, 0.070) while SeqTag achieves its highest score on B (0.726). For editing, the interaction ($\eta^2 = 5.1\%$) reflects TagFill’s Scheme B collapse (0.084) versus its competitive performance on other schemes. Encoding explains substantially more variance in correction (15.4%) than in editing (5.0%), consistent with the main text finding that encoding is a stronger design lever for precision-demanding tasks.

N Subjective Evaluation Details

We recruited 33 evaluators from university students (34% no music training, 42% hobby-level, 18% formally trained 3+ years, 6% music majors; 55% play an instrument; 52% listen daily). Systems were presented in randomized blind order via a web interface with piano roll visualizations and audio playback. Each evaluator provided 228 ratings across four experiments on three dimensions: music quality (MQ), naturalness (NA), and edit accuracy (EA), using a 1–5 MOS scale. Tables 17–20 report per-dimension scores.

Reliability and significance. Inter-rater agreement, computed as average-measures ICC(2,k), ranges from 0.75 to 0.95 across the four experiments (mean 0.87), i.e. good to excellent. For each experiment and dimension, a Friedman omnibus test rejects the null hypothesis of equal system ratings (all $p \leq .001$). Holm-corrected pairwise Wilcoxon signed-rank tests confirm the main comparisons: BEAT-C outperforms all alternative encodings ($p < .001$); on error correction, SeqTag and IterEdit outperform all generative baselines ($p < .05$); and TagFill leads on segment completion ($p < .01$).

O Efficiency Analysis

Table 21 reports per-task latency for all methods. Single-pass methods (CMLM, SeqTag, TagFill) are stable across tasks (37–65 ms). IterEdit varies substantially: completion requires few iterations (82 ms) while editing requires many (655 ms). AR latency depends on the generation strategy and output length.

Architecture-agnostic FLOPs. Wall-clock latency is affected by implementation and model size. For an implementation-independent view, Table 22 reports analytically estimated single-sample inference FLOPs (standard transformer accounting, equal sequence length 2,048). Editing methods complete inference in 1–3 forward passes (172–517 GFLOPs). Notably, AR totals only 789 GFLOPs thanks to KV caching, yet is two orders of magnitude slower in wall-clock time because its 2,048 decoding steps must execute strictly sequentially; D3PM requires 30 denoising passes (5,604 GFLOPs). The efficiency advantage of editing is thus paradigmatic—few parallel passes—rather than an artifact of implementation-level optimization.

We further examine how encoding scheme affects inference speed within the SeqTag pipeline.

O.1 Per-Encoding Efficiency

Table 23 further examines how encoding scheme affects inference speed within the SeqTag pipeline ($n \approx 20$ samples per scheme, seed 42).

Bundled encodings (C, D) are 35–40% faster than separated encodings (A, B) because bundled beats produce shorter token sequences, reducing both self-attention cost and iteration count (converging in exactly 2 iterations vs. a mean of 2.1 for separated). The p95 tail latency of Scheme B (112 ms) is notably higher than other schemes, consistent with its longer average token length (1 445 vs. 1 096–1 382 tokens). Memory usage is higher for bundled schemes (~380 MB peak vs. ~200 MB) due to the larger vocabulary, but all configurations remain well within single-GPU capacity. Combined with the cross-method results above, the overall picture is clear: edit-based methods converge in 2–3 constant-time iterations regardless of

Table 15: Key pairwise comparisons (beat exact match, 10,000 bootstrap resamples). Significance at $\alpha = 0.05$.

Comparison (A vs. B)	Task	Sch.	Mean A	Mean B	Δ	p
IterEdit ^A vs. SeqTag ^B	Corr.	—	.719	.726	−.007	.489
IterEdit vs. SeqTag	Edit.	A	.458	.382	+.076	<.001
IterEdit vs. SeqTag	Edit.	D	.480	.399	+.081	<.001
SeqTag vs. TagFill	Corr.	A	.700	.368	+.332	<.001
SeqTag vs. TagFill	Corr.	D	.700	.329	+.371	<.001
IterEdit vs. TagFill	Edit.	A	.458	.321	+.137	<.001
IterEdit vs. TagFill	Edit.	D	.480	.350	+.130	<.001
TagFill vs. No-Edit	Compl.	A	.436	.000	+.436	<.001
TagFill vs. No-Edit	Compl.	D	.326	.000	+.326	<.001

Table 16: Two-way ANOVA: encoding $\times$ method interaction on beat exact match ($n = 200$ per cell). All interactions are significant at $\alpha = 0.05$.

Analysis	Factor	F	p	η^2
Corr.: 3 methods $\times$ 4 enc (A,B,C,D)	Method	271.6	<0.001	34.6%
	Encoding	80.4	<0.001	15.4%
	Interaction	32.7	<0.001	12.5%
Edit.: 3 methods $\times$ 4 enc (A,B,C,D)	Method	68.8	<0.001	18.7%
	Encoding	18.4	<0.001	5.0%
	Interaction	6.3	<0.001	5.1%

Table 17: Exp. 1: Error correction — encoding comparison.

System	MQ	NA	EA	Avg
BEAT-C + SeqTag	4.19	4.23	4.29	4.24
REMI + SeqTag	2.99	2.64	2.78	2.80
CPWord + SeqTag	2.81	2.60	2.67	2.69
Structured + SeqTag	2.51	2.35	2.47	2.45

Table 18: Exp. 2: Error correction — method comparison.

System	MQ	NA	EA	Avg
SeqTag	4.18	4.23	4.23	4.21
IterEdit	4.14	4.15	4.09	4.13
TagFill	3.32	3.55	3.64	3.51
Diffusion SDEdit	3.40	3.23	3.11	3.25
BERT-CMLM	3.37	3.06	3.31	3.25
AR-Detect	2.95	2.99	3.19	3.05

Table 19: Exp. 3: Segment completion.

System	MQ	NA	EA	Avg
TagFill	4.10	4.17	4.17	4.15
IterEdit	3.25	3.45	3.56	3.42
LLaMA-Selective	3.38	3.02	3.07	3.15
Anticipatory	3.05	2.68	2.86	2.86

Table 20: Exp. 4: Accompaniment editing.

System	MQ	NA	EA	Avg
IterEdit	3.96	4.14	4.12	4.07
SeqTag	3.79	3.92	4.03	3.91
TagFill	3.80	3.89	3.80	3.83
LLaMA-Selective	3.45	3.08	3.09	3.21
BERT-CMLM	3.23	3.08	3.12	3.14
Diffusion SDEdit	3.08	2.84	2.90	2.94

Table 21: Per-task inference latency (ms) and parameters.

Method	Corr.	Edit.	Compl.	Params (M)
BERT-CMLM	40	42	37	30.2
SeqTag	65	57	—	37.2
TagFill	60	60	60	63.8
IterEdit	273	655	82	33.9
Diffusion r03	445	484	434	34.4
Anticipatory	—	—	409	128.1
AR-Selective	9,357	—	—	162.0
AR-Detect	9,677	9,458	—	199.2
AR-Teacher	12,634	—	—	162.0
AR-Prompt	22,934	—	23,654	162.0

Table 22: Estimated single-sample inference FLOPs at equal sequence length 2,048. Forward passes are the dominant factor.

Method	Forward passes	GFLOPs
SeqTag	1	172
BERT-CMLM	1	172
TagFill	3 (1 tag + 2 fill)	517
IterEdit	~3	517
Diffusion (D3PM)	30	5,604
AR (LLaMA)	2,048 (sequential)	789

sequence length, with bundled encoding providing an additional speed advantage at the cost of higher memory.

Table 23: Inference efficiency by encoding scheme (SeqTag pipeline).

Metric	A	B	C	D
Time / sample (ms)	51.4	54.9	38.3	37.6
Samples / sec	19.5	18.2	26.1	26.6
Mean iterations	2.12	2.11	2.00	2.00
p95 latency (ms)	83.5	112.3	72.3	71.8
Model memory (MB)	101	101	142	142
Peak memory (MB)	203	211	380	380

P Multi-Track Generalization Details

The multi-track dataset comprises 108,055 pieces from the Lakh MIDI Dataset, covering 105 General MIDI programs (Piano 19.0%, Drums 13.3%, Electric Bass 4.5%, Strings 4.4%, Guitar 11.1% combined). Pieces contain 2–10 tracks (mean 6.2, median 6) and 17–243 measures (mean 86.6). Tracks are merged into two voices and encoded with Scheme C ($|V|=7,145$), with 64.6% of sequences exceeding the 2,048-token limit and requiring truncation at bar boundaries.

Table 24 provides per-task breakdowns. SeqTag on correction achieves 0.258 beat exact match with notably high nf1 (0.797) and low MPE (3.57), suggesting that multi-track accompaniment has more predictable pitch patterns than piano. TagFill on completion reaches 0.271 beat exact match with FMD of 0.49—the lowest among all task–setting combinations—indicating high distributional quality. IterEdit on editing shows a beat drop (0.269→0.196) with comparable nf1 (0.435→0.534).

Table 24: Multi-track generalization: full metrics per task (Scheme C, perturbed beats only). Piano values under the same Scheme C for reference.

Task	Setting	beat↑	nf1↑	cf1↑
Corr.	Piano No-Edit	.035	.435	.452
	Piano SeqTag	.400	.531	.546
	Multi No-Edit	.000	.803	.798
	Multi SeqTag	.258	.797	.804
Compl.	Piano No-Edit	.000	.000	.000
	Piano TagFill	.298	.442	.507
	Multi No-Edit	.028	.000	.000
	Multi TagFill	.271	.494	.628
Edit.	Piano No-Edit	.035	.435	.452
	Piano IterEdit	.269	.435	.448
	Multi No-Edit	.003	.539	.565
	Multi IterEdit	.196	.534	.568

A consistent pattern across all three tasks: beat exact match is lower on multi-track data (reflecting greater diversity), while note-level metrics (nf1, chroma_f1) are comparable or better (reflecting pitch redundancy in ensemble arrangements). The framework transfers without architectural modification, confirming that the BEAT encoding’s structural properties generalize beyond the piano domain.

References

- [1] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. 2021. Structured Denoising Diffusion Models in Discrete State-Spaces. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34.